ANLP HW3 Project Proposal

Wenguang Dong
Carnegie Mellon University
wenguand@andrew.cmu.edu

Yichen Ji
Carnegie Mellon University
yichenj@andrew.cmu.edu

Juan Pablo Naranjo
Carnegie Mellon University
jpnaranj@andrew.cmu.edu

1 Motivation

While Hypothetical Document Embedding (HyDE) (Gao et al., 2023) improves zero-shot dense retrieval by generating a hypothetical document from the query and retrieving real documents near that embedding, the method's heavy dependence on the quality of the generated hypothetical text can make the retrieval process unstable. The HyDE paper itself notes that the hypothetical document is "unreal" and may contain false details (Gao et al., 2023). If the hypothetical answer is poorly grounded, retrieval can drift toward an incorrect neighborhood in embedding space, which may mislead downstream retrieval and answer generation. This instability becomes more consequential in query-focused summarization (Zhong et al., 2021) and long-form question answering (Fan et al., 2019), where retrieval must support multi-aspect reasoning, broad evidence coverage, and coherent long-form generation rather than pinpoint fact lookup.

Our idea is to experiment with two complementary ways of improving the reliability of this intermediate representation. First, we use a knowledge graph (KG) to inject related entities and relations while generating the hypothetical answer, making it more structurally grounded and semantically constrained, in the spirit of graph-based retrieval methods such as GraphRAG (Edge et al., 2025). Second, instead of asking the language model to generate a full hypothetical document, we follow the intuition of RAPTOR (Sarthi et al., 2024) and ask it to produce a hypothetical summary: a shorter, higher-level abstraction that preserves salient concepts while reducing hallucinated detail.

Based on these ideas, our research is motivated by the hypothesis that KG-grounded, summary-level hypothetical representations may make HyDE-style retrieval more stable and less prone to deviating from the fact than free-form hypothetical documents.

2 Literature Review

2.1 Hierarchical Indices

Traditional RAG (Lewis et al., 2020) relies on flat chunking (e.g., splitting a PDF into 500-token blocks). While easy to implement, it causes "semantic fragmentation". If an answer spans multiple chunks or requires document-level understanding, flat retrieval fails. The current research landscape is actively shifting toward **Hierarchical RAG**, which organizes knowledge into multi-level trees or graphs to allow dynamic, multi-resolution retrieval.

(Sarthi et al., 2024) proposed RAPTOR, an approach for hierarchical RAG. RAPTOR clusters text chunks and uses an LLM to summarize those clusters recursively, building a bottom-up tree. During retrieval, it matches queries against both high-level summaries and low-level chunks. Expanding on this trend of structured retrieval, Microsoft recently introduced GraphRAG (Edge et al., 2025), which merges hierarchical indexing with knowledge graphs. GraphRAG extracts entities from text to build a graph, and then uses community detection algorithms to cluster those entities into a multi-level hierarchy, generating summaries for each community layer.

A shortcoming of these hierarchical approaches (whether tree-based like RAPTOR or graph-based like GraphRAG) is that they rely heavily on the user's query matching the embedding of a high-level summary to successfully traverse down the index. If a query is poorly phrased, traversal fails at the root. HyDE (Gao et al., 2023) solves query phrasing, but it generates a flat, detailed hypothetical document. This hypothetical document embedding will not match optimally with the abstract, high-level summaries at the top tiers of a hierarchical index. There is a good opportunity to adapt HyDE to generate multi-granularity representations that perfectly align with these hierarchical data

structures.

2.2 Knowledge Graph

GraphRAG is introduced to solve global, corpus-level questions such as identifying major themes, recurring patterns, or cross-document relationships. Such questions are often categorized as query-focused summarization or sense-making problems (Klein et al., 2006) that require understanding of the entire corpus.

GraphRAG works by turning the corpus into a hierarchical knowledge graph by creating entities and relationships for each chunk, then merging them into a graph where nodes represent entities and edges represent relations using LLM (Ban et al., 2023). The system then applies hierarchical community detection to group strongly connected entities into communities at multiple levels. Each community is summarized, and higher-level summaries are built recursively from lower-level ones. At query time, GraphRAG uses a map-reduce process: different community summaries generate partial answers, which are then combined into a final global answer. This helps improve performance on general questions because the model reasons over a structured, multi-level summary of the whole corpus rather than a few retrieved chunks.

Compared with traditional vector RAG, GraphRAG produces more comprehensive and diverse answers on global questions. However, vector RAG remained better in directness, meaning its answers were usually shorter and more targeted. Overall, GraphRAG is more suitable for whole-corpus synthesis, while vector RAG remains stronger for narrow fact retrieval.

2.3 HyDE

Research on hypothetical techniques in retrieval-augmented generation (RAG) addresses a central retrieval problem: the user query and the target document chunk often live in different semantic forms. A short question may not share enough lexical or embedding-level overlap with the chunk that actually contains the answer. Recent work tackles this mismatch by asking a language model to generate an intermediate representation before retrieval or reranking. This representation may be a hypothetical document, hypothetical answer, hypothetical question, or hypothetical prompt. Across the literature, the shared intuition is that generated text can better expose the latent information need and therefore align the query with relevant corpus content more effectively than the raw query alone.

The most influential work in this area is HyDE (Hypothetical Document Embeddings). HyDE first prompts an instruction-following LLM to generate a hypothetical document for a query, then embeds that generated document and uses the embedding to retrieve real corpus documents (Gao et al., 2023). HyDE showed strong zero-shot retrieval performance in multiple tasks and languages.

Another important concept is HyPE (Hypothetical Prompt Embeddings). Instead of generating hypothetical content at query time, HyPE moves this process to the indexing stage. For each chunk, it precomputes multiple hypothetical prompts/questions and indexes embeddings based on those generated prompts, effectively reframing retrieval as question-to-question matching rather than question-to-document matching(Vake et al., 2025). Compared with HyDE, HyPE reduces runtime overhead because it avoids synthetic generation during inference, and it represents an important trend from online hypothetical generation** toward offline corpus-side hypothetical augmentation.

A third especially relevant case is hypothetical answer: instead of only generating a pseudo-document for retrieval, Related article uses a provisional answer-like intermediate signal inside a structured reasoning pipeline (Zhang et al., 2025). Conceptually, this differs from HyDE and HyPE because the hypothetical text is not mainly used to bridge a query–document style mismatch at the retrieval interface; rather, it supports **reasoning decomposition and intermediate verification** in multi-hop QA.

Several broad patterns emerge from prior work on hypothetical retrieval in RAG. First, the field is moving beyond simple lexical expansion toward representation engineering, where generated text reshapes the semantic space between queries and documents. Second, there is a shift from online per-query generation to offline indexing-time generation, which reduces inference-time latency. Third, researchers increasingly emphasize corpus-grounded hypotheticals rather than relying only on the LLM's internal knowledge. Within this trend, TreeQA's use of hypothetical answers is especially promising. Compared with HyDE, which generates an entire hypothetical document for each query, TreeQA pairs each sub-question with a specific, verifiable hypothetical answer, making the intermediate representation lighter, more focused, and more directly aligned with the query. However, this

approach still leaves substantial room for improvement. TreeQA relies on a tree structure, which supports hierarchical reasoning but cannot naturally represent shared intermediate nodes or cross-branch dependencies common in real multi-hop reasoning; this motivates extending the framework to a DAG or graph structure. In addition, its self-correction mechanism is still largely post hoc. The paper's error analysis shows that retrieval failure is the main source of errors, while incorrect tree construction can also prevent contradictions from being properly propagated. Therefore, a promising research direction is to retain the efficiency of hypothetical answers while improving graph-based reasoning and strengthening self-correction, especially the update logic triggered when retrieved evidence contradicts the current intermediate answer.

3 Initial Proposed Approaches

3.1 Initial Proposed Approach #1

Instead of using HyDE to generate a full, detailed hypothetical document, we propose to prompt the LLM to generate a hypothetical high-level **summary** of the answer. We would then take this "fake summary" to search the top-tier summary nodes in the hierarchical index. Once the most relevant top-tier nodes are identified, we can then traverse down to the leaf chunks.

3.1.1 Compute Requirements Estimate

Moderate to High. Indexing will require substantial API calls to generate the tree summaries (e.g., using GPT-4o-mini or Llama-3). Inference will require 1-2 extra LLM calls per query to generate the hypothetical text, plus standard embedding and vector similarity compute.

3.2 Initial Proposed Approach #2

Our proposed KG-based approach improves HyDE by grounding the hypothetical representation before retrieval. Given a query, we first extract key entities and concepts, then use a knowledge graph to retrieve related entities and relations as structured context. Instead of prompting the LM to write a fully free-form hypothetical document, we ask it to generate a grounded hypothetical summary constrained by this KG evidence. The summary is then embedded and used for dense retrieval, following the HyDE pipeline. This design aims to reduce hallucinated details and stabilize retrieval by anchoring the hypothetical text to explicit entity-relation structure.

3.2.1 Compute Requirements Estimate

The compute requirement is relatively light because no model training is involved. OpenAI API will be used for entity and relationship extraction. Neo4j will be used for building KG, and it mainly relies on CPU and memory for graph construction and querying. Therefore, the main cost of this pipeline is more likely to come from API usage and runtime rather than heavy local compute.

4 Initial Proposed Data

We plan to use three complementary datasets: RAGBench(Friel et al., 2025), QMSum (Zhong et al., 2021), and ELI5 (Fan et al., 2019). RAGBench is a standard and popular benchmark designed for evaluating retrieval-augmented generation systems, containing diverse question types that require accurate retrieval and faithful generation, making it suitable for assessing both retrieval quality and answer faithfulness. QMSum is a query-focused meeting summarization dataset, where answering questions requires aggregating information across long transcripts, making it useful for testing our system's reasoning and long-context retrieval. ELI5 consists of open-domain, explanatory questions from Reddit, with long-form answers that emphasize reasoning and knowledge synthesis. Together, these datasets cover factual QA, long-document reasoning, and explanatory generation, enabling comprehensive evaluation of our method's retrieval robustness and reasoning ability.

5 Initial Proposed Evaluation

The theoretical constructs that are most relevant to evaluate our method are **retrieval accuracy** (did you find the right nodes in the tree?) and **generation quality** (did the LLM synthesize a good answer without hallucinating?). For retrieval accuracy, some proxies to evaluate if the correct ground-truth evidence chunks were retrieved from the hierarchy are Recall@k or Mean Reciprocal Rank. As for generation quality, using the RAGAS framework (automated LLM-as-a-judge metric system) we could measure *context precision*, *faithfulness* and *answer relevance*.

References

Taiyu Ban, Lyvzhou Chen, Xiangyu Wang, and Huanhuan Chen. 2023. From query tools to causal architects: Harnessing large language models for advanced causal discovery from data. *arXiv preprint arXiv:2306.16902*.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2025. From local to global: A graph rag approach to query-focused summarization. *Preprint*, arXiv:2404.16130.

Angela Fan, Yacine Jernite, Ethan Perez, David Grangier, Jason Weston, and Michael Auli. 2019. ELI5: Long form question answering. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3558–3567, Florence, Italy. Association for Computational Linguistics.

Robert Friel, Masha Belyi, and Atindriyo Sanyal. 2025. Ragbench: Explainable benchmark for retrieval-augmented generation systems. *Preprint*, arXiv:2407.11005.

Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. 2023. Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Toronto, Canada. Association for Computational Linguistics.

G. Klein, B. Moon, and R.R. Hoffman. 2006. Making sense of sensemaking 1: Alternative perspectives. *IEEE Intelligent Systems*, 21(4):70–73.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc.

Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. 2024. Raptor: Recursive abstractive processing for tree-organized retrieval. *Preprint*, arXiv:2401.18059.

Domen Vake, Jernej Vičič, and Aleksandar Tošić. 2025. Bridging the question–answer gap in retrieval-augmented generation: Hypothetical prompt embeddings. *IEEE Access*, 13:129952–129961.

Xiangrui Zhang, Fuyong Zhao, Yutian Liu, Panfeng Chen, Yanhao Wang, Xiaohua Wang, Dan Ma, Huarong Xu, Mei Chen, and Hui Li. 2025. Treeqa: Enhanced llm-rag with logic tree reasoning for reliable and interpretable multi-hop question answering. *Knowledge-Based Systems*, 330:114526.

Ming Zhong, Da Yin, Tao Yu, Ahmad Zaidi, Mutethia Mutuma, Rahul Jha, Ahmed Hassan Awadallah, Asli Celikyilmaz, Yang Liu, Xipeng Qiu, and Dragomir Radev. 2021. QMSum: A new benchmark for query-based multi-domain meeting summarization. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5905–5921, Online. Association for Computational Linguistics.